

1. Motivation	3
Content	3
2. Configuration Space	3
2.1. Degrees of freedom	3
2.1. From workspace to configuration space	4
2.2. Paths	5
2.3. Obstacles	5
3. Bug algorithms	6
3.1. Base Navigation	6
3.2. Bug algorithms	6
3.3. Application of Bug Strategies for Configuration Spaces Ch 3.2.2	7
4. Classical path planning approaches	7
4.1. Roadmaps Ch 5	7
4.1.1. Visibility graph method CH. 5.1	7
4.1.2. Voronoi Diagram CH. 5.2	8
4.2. Cell Decomposition Ch 6	8
4.2.1. Trapezoidal decomposition Ch 6.1	8
4.2.2. The Halting Problem	9
4.2.3. Implementation of Vertical Cell Decomposition in a higher dimension 6.3.3	9
4.2.4. k-d Trees	10
4.2.5. Approximate Cell decomposition	10
4.3. Potential field	10
4.3.1. Function of the potential field	11
4.3.2. Bushfire Algorithm Ch 4.3.2	12
4.3.3. Wave Front planner CH 4.5	12
5. Sampling-Based Algorithms Ch7	13
5.1. Characteristics of Sampling-Based Planners	13
5.2. Sampling Strategy: Create a uniform distribution	13
5.3. Classic multiple query PRM (Probabilistic Roadmap)	13
5.4. Single Query PRM Ch7.2	15
5.4.1. rapidly-exploring random trees (RRT; by LaValle & Kuffner) Ch 7.2.2	15
5.5. Analysis of PRM: Expansive Spaces Ch 7.4.2	16
5.6. PRM Conclusion	18
5.7. OBPRM: Obstacle-Based PRM	18
6. Kalman Filtering Ch 8	19
6.1. Data Fusion example	19
6.2. Observability matrix	20
6.3. Problems with the Kalman filter	21
6.4. Kalman examples	21
6.4.1. Example: Water level in tank	22
6.5. The Extended Kalman Filter (EKF) Ch 8.3	22
6.6. Linearized Motion Model for a Robot	24
6.7. Extended Kalman Filter for SLAM Ch 8.4	24

6.8. Summary	25
7. Unscented Kalman Filter	26
7.1. Filtering Problem: General Problem Statement	26
7.2. Unscented Transformation	26
7.3. Scaled Unscented Transformation (SUT)	27
7.4. Unscented Kalman Filter	27
8. Bayesian and Particle Filter Ch 9	28
8.1. Localization	28
8.2. Bayesian Filtering	29
8.3. Particle Filtering	30
9. Overview Filters	32

1. Motivation

CONTENT

- Complete knowledge
 - Bug Algorithms
 - Configuration Space
 - Potential Functions
- Offline preprocessing
 - Road maps
 - Cell Decompositions
- Probabilistic Methods
 - Sampling-Based Algorithms
 - Kalman Filtering
- Filtering of Uncertainty
 - Bayesian Methods
 - Dynamic Environments

Planning algorithms are even impacting fields as far away from robotics as computational biology. Two major problems are protein folding and drug design. Such molecules are generally flexible. The *docking problem* involves determining whether a flexible molecule can insert itself into a protein cavity. The assembly problem and can be solved by motion planning algorithms.

2. Configuration Space

- Task space: a space where the robot's task can be naturally expressed (eg: entire hall $\mathbb{R}^3 \times S^1$)
- Workspace: captures the „reachable“ space of the end effector (eg: reachable points in $\mathbb{R}^3$)
- Both involve some user choice and often are different from Configuration space
- Rigid body: one-piece robot
- Link: rigid piece, often a part of multi-piece robot
- Links of a multi-piece robot are joined with joints (connectors)

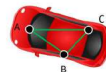


2.1. DEGREES OF FREEDOM

- Configuration: specification of where all points of a robot are
- Degrees of freedom (DOF): smallest number of real-valued (i.e. continuous) coordinates to fully describe configurations of robots

Single body

- DOF = total DOF of points - # of constraints
- Example Car: $2 \times 3 - 3 = 3$ (each of the three points on the car can move in x and y orientation, 3 constraints: $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow A$)



General robots

- 2D Chains: Base link is 3D, if fixed, then often 1D, Adding joints generally adds one more dimension
- 3D Chains: Base link is 6D, if fixed, depending on the joint, Then add the DOF of each additional joint
- Closed Chains: also known as a parallel mechanism, is one where the links form one or more closed loops
 - If the mechanism has k links, then one is designated as a stationary "ground" link, and $k - 1$ links are movable
 - Each movable link has N degrees of freedom: $N = 6$ for spatial mechanism, $N = 3$ for planar
 - Each of the n joints places $N - f_j$ constraints on the feasible motion

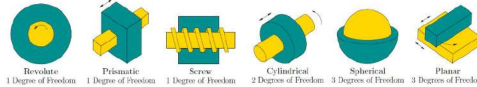
Grübler's Formula: $DOF = N(k-1) - \sum_{i=1}^n (N-f_i) = N(k-n-1) + \sum_{i=1}^n f_i$

- N: 6 for 3D, 3 for 2D
- k: # of links (including the ground link)
- n: # of joints
- f_i : DOF of the joint



Type of joints

- 2D: Revolute and Prismatic
- 3D: see picture

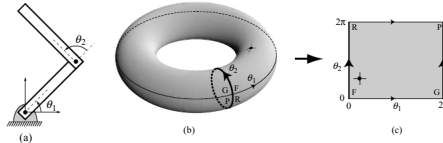


2.1. FROM WORKSPACE TO CONFIGURATION SPACE

- **Configuration of a moving object:**
 - specification of the position of every point on the object $\rightarrow$ vector of position & orientation
 - $q = (q_1, q_2, \dots, q_n)$
- **Configuration space / C-space:**
 - set of all possible configurations of the system $\rightarrow$ configuration is simply a point in this abstract configuration space C
 - The number of degrees of freedom of a robot system is the dimension of the configuration space, or the minimum number of parameters needed to specify the configuration.
 - powerful abstraction for solving motion planning problems (finding feasible motions for robots from x_{-1} to x_{-G})
 - in C-space: searching for a path in the joint space of 2D position $(x, y) \in \mathbb{R}^2$ and rotation $\theta \in S^1$
 - topology of C: usually not Cartesian

Examples

- screw in nut is 1D because height is coupled to rotation
- (a) Workspace: A two-joint manipulator.
- (b) The configuration of the robot is represented as a point on the toral configuration space.
- (c) Configurations space: The torus can be cut and flattened onto the plane. This planar representation has "wraparound" features where the edge FR is connected to GP , etc.
- many more on slides



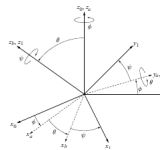
Matrix representations in $SO(3)$

- nine elements of rotation matrix $R \in SO(3)$ are subject to six constraints $\rightarrow$ 3 DOF
- Expectation: $SO(3)$ can be parametrized using three variables $\rightarrow$ Euler angles are often used
- Correspond to three successive rotations around z-axis, y-axis and lastly x-axis:

$$R = R_{z,\phi} R_{y,\theta} R_{x,\psi}$$

$$= \begin{bmatrix} c_\phi & -s_\phi & 0 \\ s_\phi & c_\phi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} c_\theta & 0 & s_\theta \\ 0 & 1 & 0 \\ -s_\theta & 0 & c_\theta \end{bmatrix} \begin{bmatrix} c_\psi & -s_\psi & 0 \\ s_\psi & c_\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} c_\phi c_\theta c_\psi - s_\phi s_\psi & -c_\phi c_\theta s_\psi - s_\phi c_\psi & c_\phi s_\theta \\ s_\phi c_\theta c_\psi + c_\phi s_\psi & -s_\phi c_\theta s_\psi + c_\phi c_\psi & s_\phi s_\theta \\ -s_\phi c_\psi & s_\phi s_\psi & c_\phi \end{bmatrix}$$



- no global parametrization because there are some singularities $\rightarrow$ define functions like atan, ...

Axis-angle parametrization: Rodrigues formula

- efficient algorithm for rotating a vector in space, given an axis and angle of rotation
- can be used to transform all three basis vectors to compute a rotation matrix in $SO(3)$, the group of all rotation matrices, from an axis-angle representation
- In other words, the Rodrigues' formula provides an algorithm to compute the exponential map from $so(3)$, the Lie algebra of $SO(3)$, to $SO(3)$ without actually computing the full matrix exponential

$$\mathbf{R} = \mathbf{I} + (\sin \theta) \mathbf{K} + (1 - \cos \theta) \mathbf{K}^2$$

is the rotation matrix through an angle θ counterclockwise about the axis $\mathbf{k}$.
 I the 3×3 identity matrix

Unit quaternion (p.494)

- The axis-angle parameterization described above parameterizes a rotation matrix by three parameters. Quaternions, which are closely related to the axis-angle parameterization, can be used to define a rotation by four numbers.
- Quaternions are a popular choice for the representation of rotations in three dimensions because compact, no singularity and naturally reflect the topology to the space of orientations

$$\mathbf{u} = (u_1, u_2, u_3, u_4) \quad \text{with } u_1^2 + u_2^2 + u_3^2 + u_4^2 = 1$$

$$(u_1, u_2, u_3, u_4) = \left(\cos \theta/2, n_x \sin \theta/2, n_y \sin \theta/2, n_z \sin \theta/2 \right) \quad \text{with } n_x^2 + n_y^2 + n_z^2 = 1$$

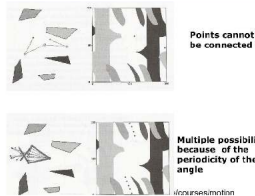
- # DOF = 6
- Topology = $\mathbb{R}^3 \times SO(3)$

2.2. PATHS

- path: continuous curve connecting two configurations q and q' :
 $\tau : s \in [0, 1] \rightarrow \tau(s) \in C$ such that $\tau(0) = q$ and $\tau(1) = q'$.
- trajectory: path parametrized by time: $\tau : t \in [0, T] \rightarrow \tau(t) \in C$
- Constraints: Finite length, Bounded curvature, Smoothness, Minimum length, time, energy, ...

Free space topology

- A free path lies entirely in the free space $F \rightarrow$ no contact between robot and obstacle
- The moving object and the obstacles are modeled as closed subsets, meaning that they contain their boundaries.
- C-obstacles are closed subsets of the configuration space C
- Free space F is an open subset of C . Hence, each free configuration is the center of a ball of non-zero radius entirely contained in F .

**Semi-free space**

- semi-free path $\rightarrow$ allows robot to contact the boundary of the obstacle
- semi-free space is a closed subset of C . Its boundary is a superset of the boundary of F .

2.3. OBSTACLES

- A configuration q is collision-free, or free, if a moving object placed at q does not intersect any obstacles in the workspace.
- The free space F is the set of free configurations.
- A configuration space obstacle (C-obstacle) is the set of configurations where the moving object collides with workspace obstacles.
- Is the configuration in the free space? Compute the position of the robot and check for collision with any obstacle at that position:
 - if colliding $\rightarrow$ configuration within C-space obstacle

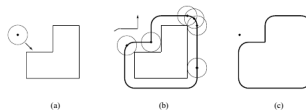


Figure 3.4 (a) The circular mobile robot approaches the workspace obstacle. (b) By sliding the mobile robot around the obstacle and keeping track of the curve traced out by the reference point, we construct the configuration space obstacle. (c) Motion planning for the robot in the workspace representation in (a) has been transformed into motion planning for a point robot in the configuration space.

- otherwise $\rightarrow$ free space

3. Bug algorithms

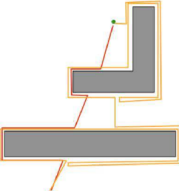
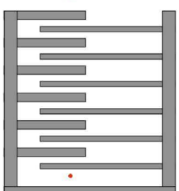
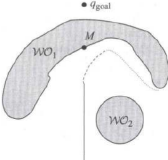
3.1. BASE NAVIGATION

- What is the simplest policy to perform navigation?
 - Random Walk: Move in a random direction until you hit something, then you go in a new direction, stop when you get to the goal, assuming it can be recognized
- What is a simple deliberative policy

3.2. BUG ALGORITHMS

- Assumption: bounded world W
- Known: global goal, measurable distance $d(x,y)$
- Unknown: obstacle WO_i
- Local sensing: tactile (e.g. bump sensor), distance travelled (odometry: use of data from motion sensors to estimate change in position over time)
- Navigation: path start from q_s to goal q_d , as a sequence of hit/leave point pairs on obstacles (q^H , q^L)

Name	Description	Working Case	Problem case
Bug 0	• Head towards goal • When hit point set, follow wall until you can move towards goal again (leave point) • continue from 1		
Bug 1	• Head towards goal • When hit obstacle, circumnavigate, setting leave point as closest to goal • Return to leave point • if bump current obstacle, return fails, else, continue from 1 • Best case: $d(q_{start}, q)$ • Worst case (n obstacles, p_i: perimeter): $L_{Bug1} \leq d(q_{start}, q) + 1.5 \cdot \sum_{i=1}^n p_i$		no path exists: line(q^L, q_d) intersects current obstacle failure bump occurs immediately not possible
Bug 2	• Head towards goal on m-line • When hit point set, traverse obstacle until m-line is encountered • Set leave point and exit obstacle • Continue from 1 • Duration (intersects i-th obstacle n_i times): $L_{Bug2} \leq d(q_{start}, q) + 0.5 \cdot \sum_{i=1}^n n_i p_i$	m-line: straight line path to goal	$n = 2$ here

Name	Description	Working Case	Problem case
Comparison			
Tangential Bug (Page 23)	- improvement to Bug 2: shorter path using a range sensor with a 360° infinite orientation resolution - Moves on straight line towards the goal until obstacle is sensed - Moves along the obstacle in distance of sensing radius		

3.3. APPLICATION OF BUG STRATEGIES FOR CONFIGURATION SPACES CH 3.2.2

- grid-based representation of the configuration space
- grid on the torus: for each point on the grid, perform collision check
- left: two-joint arm in a planar workspace, right: configuration space

4. Classical path planning approaches

- Construct data structure once and then use data structure to plan subsequent paths more quickly
- Topological maps** $\rightarrow$ **Roadmaps**: Represent the connectivity of the free space by a network of 1-D curves (connected nodes) and edges represent an adjacency relationship between nodes
- Geometric models** $\rightarrow$ **Cell Decomposition**: Decompose the free space into simple cells and represent the connectivity of the free space by the adjacency graph of these cells
- Occupancy grids** $\rightarrow$ **Potential field**: Define a potential function over the free space that has a global minimum at the goal and follow the steepest descent of the potential function

4.1. ROADMAPS CH 5

- A union of one-dimensional curves is a roadmap if for all q_{start} and q_{goal} in free space that can be connected by a path, the following properties hold: Accessibility, Departability, Connectivity

4.1.1. VISIBILITY GRAPH METHOD CH. 5.1

- Defining characteristics: nodes share an edge if they are within line of sight of each other and all points in the robot's free space are within line of sight of at least one node on the visibility map $\rightarrow$ properties of accessibility and departability, but connectivity must be proven
- defined in a 2-D polygonal configuration space
- Nodes: v_i of the visibility graph include the start q_{start} and goal location q_{goal} and



Edges: 18 ppt



for two obstacles
4 lines



all the vertices of the configuration space obstacles (not part of the offline graph, only added online)

- Graph edges: are straight lines that connect two line-of-sight nodes

Algorithm

Input: q_{init} , q_{goal} , polygonal obstacles

Output: visibility graph G

```

for every pair of nodes  $u, v$ 
    if segment  $(u, v)$  is an obstacle edge then insert edge  $(u, v)$  into  $G$ ;
    else
        for every obstacle edge  $e$ 
            if segment  $(u, v)$  intersects  $e$ 
                go to (1);
        insert edge  $(uv)$  into  $G$ .
  
```

- Computation time in $O(n^2)$ space:
 - Simple algorithm: $O(n^3)$
 - More efficient: Rotational Sweep $O(n^2 \log(n))$, Optimal $O(n^2)$

Simplified Visibility Graph

- all edges that do not lie on a supporting or separating line are removed
- It's beneficial because it has fewer edges making the search for the shortest path more efficient
- A supporting line is tangent to two obstacles such that both obstacles lie on the same side of the line.
- A separating line is tangent to two obstacles such that the obstacles lie on opposite sides of the line.

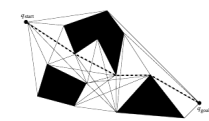
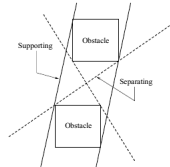


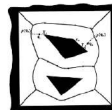
Figure 5.4 The thin solid lines delineate the edges of the visibility graph for the three obstacles represented as filled polygons. The thick dashed line represents the shortest path between the start and goal.



Figure 5.6 Reduced visibility graph.

4.1.2. VORONOI DIAGRAM CH. 5.2

- Generate paths that maximizes clearance: set of points where the distance to the two closest obstacle is the same
- applicable mostly to 2-D configuration spaces
- Space $O(n) \rightarrow$ Running time $O(n \cdot \log(n))$
- Bushfire method can be used to construct the Voronoi Diagram
 - Input: grid of zeros for free space, ones for obstacles
 - Output: discrete map with each pixel has value equal to the distance to the closest point on the closest obstacle



4.2. CELL DECOMPOSITION CH 6

- Exact cell decomposition: The free space F is represent by a collection of non-overlapping simple cells whose union is exactly F
- Examples of cells: trapezoids, triangles

4.2.1. TRAPEZOIDAL DECOMPOSITION CH 6.1

- vertical line sweeps through space: if hit corner $\rightarrow$ separation line
- connect cells by a straight line segment that does not intersect any obstacle $\rightarrow$ connecting the midpoints of the vertical extensions to the centroids of each trapezoid
- Running time: $O(n \log(n))$ by planar sweep in space $O(n)$



- mostly for 2-D configurations
- Nodes: cells
- Edges: There is an edge between every pair of nodes whose corresponding cells are adjacent
- **Boustrophedon Decomposition:** extension of algorithm to avoid unnecessary nodes (many small cells are formed that can seemingly be aggregated with neighboring cells) $\rightarrow$ keep only lines on critical points (only lines where you have to make decisions)
- **Canny's Roadmap algorithm** for round obstacles: Cell boundaries are placed at critical points, which are points where the tangential line touches the object

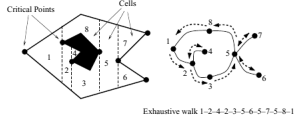


Figure 6.10 The boustrophedon decomposition of a space and its adjacency graph. The nodes represent the cells and the edges indicate the adjacent cells. An exhaustive walk on the graph is generated.

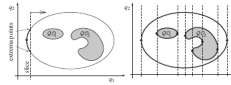


Figure 6.23 Roadmap two-dimensional environment with two obstacles (Q1 and Q2). The left figure contains a single slice, represented by a dashed line, and a partially constructed roadmap. The right figure contains the complete roadmap and slices passing through all critical points.

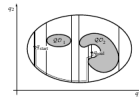


Figure 6.24 Complete roadmap curves traced out with solid lines. A path from start to goal is denoted as a thin curve.

- combinatorial motion planning $\rightarrow$ combinatorial partitioning of the environment $\rightarrow$ makes algorithms complete

4.2.2. THE HALTING PROBLEM

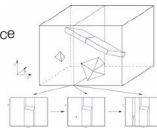
- Is it possible to determine whether a program will stop?
- This problem is undecidable by using another algorithm (a computer/turing machine cannot really tell)
- Sketch of proof (via contradiction)
 - Suppose an algorithm H_{TM} can decide whether an algorithm will halt
 - Construct a new algorithm A_{TM} and run on input A_{TM} :
 - If A_{TM} halts ($\perp$), then $H_{TM}(A_{TM}) == \text{true}$, A_{TM} should loop forever
 - If A_{TM} does not halt, then $H_{TM}(A_{TM}) == \text{false}$, A_{TM} should halt
 - Both are contradictions $\rightarrow H_{TM}$ cannot exist!
- Implications: Not all algorithms are complete $\rightarrow$ Solution: do not use a full Turing Machine to do the testing
- Combinatorial algorithms are complete (because every single point in C_{free} is covered, motivates the development to of combinatorial methods for higher dimensions)
- Combinatorial algorithms are algorithms that deal with *combinatorial structures*, which are sets, ordered n-tuples, and any structures that can be built from them, like graphs.
- Example: Piano Mover's Problem
 - Use autonomous robots to move a paint around without hitting other objects
 - Not immediately clear, if such problems are solvable
 - Various algorithms tried it and finally it was solvable with Collin's cylindrical algebraic decomposition

$A_{TM} : \text{on input algorithm } X$
 if $H_{TM}(X) == \text{true}$ then loop forever;
 else return true;

4.2.3. IMPLEMENTATION OF VERTICAL CELL DECOMPOSITION IN A HIGHER DIMENSION

6.3.3

- VCD can be extended to any dimension by recursively applying the sweeping idea
- requirements: C_{obj} is piecewise linear $\rightarrow C_{\text{obj}}$ is represented as a semi-algebraic model for which all primitives are linear (e.g. polyhedral robot that can translate in R^3 and the obstacles in W are polyhedra)
- Steps:
 1. Pick a dimension x_i
 2. **Plane Sweep:** Sweep the configuration space using a $(n-1)$ dimensional plane orthogonal to x_i axis (unique hyperplane). It produces a polygonal slice of C_{obj} (see three slices in figure).
 3. **Line Sweep:** Sweep a line along the y -axis through the subdivided



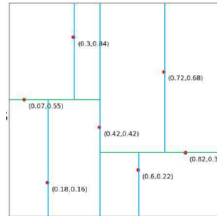
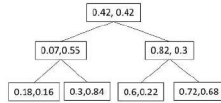
volumes and leave boundaries at critical point

4. Repeat 2. to each of the $(n-1)$ dimensional slices

- The 2-cells in a slice are actually slices of 3-cells in the 3D decomposition. The only places in which these 3-cells can critically change is when the sweeping plane stops at some x value.
- The roadmap is constructed by placing a sample point in the center of each 3-cell and 2-cell. The vertices are the sample points, and edges are added to the roadmap by connecting the sample points for each case in which a 3-cell is adjacent to a 2-cell.
- This same principle can be extended to any dimension, but the applications to motion planning are limited because the method requires linear models (or at least it is very challenging to adapt to nonlinear models; in some special cases, this can be done).
- Canny's Algorithm in higher dimensions
 - basic idea: project directed from $\mathbb{R}^n$ to $\mathbb{R}^2$ many times
 - drops computation time to singly exponential (a^n)
 - still not very practical but theoretically much better

4.2.4. K-D TREES

- A data structure for storing points in k dimensions
- Assume a tree like structure
- Useful for finding points with certain properties
- Can be used for solving 1-NN
- Construction of a k -d tree for n points
 - Pick dimension i , pick a point with coordinates $x = (x_1, \dots, x_i, \dots, x_k)$
 - Split the points based on x_i (greater or less than)
 - Repeat the above two steps recursively: Increase i (modulo k) each time (i.e., pick a new dimension each time)
 - Depth: $\log(n)$ if balanced
 - Construction takes $O(k \cdot n \log(n))$ time
 - Each dimension needs sorting $\sim O(n \log(n))$
 - Can speed up to $O(n \log(n))$
- Very efficient, but: relies on complete knowledge about environment, if higher level node is removed $\rightarrow$ reconstruct the tree again, changes in spaces $\rightarrow$ reorder everything

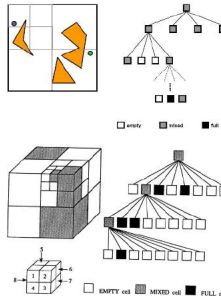


4.2.5. APPROXIMATE CELL DECOMPOSITION

- F is represented by a collection of non overlapping cells whose union is contained in F
- does not need to be balanced, adapts to the structure

Quadtree decomposition

- first node: whole space
- divide space into subspaces $\rightarrow$ subsides
- only mark if node is empty, mixed or full
- graph only extends to lower entries if it has a mixed entry
- if obstacle is removed from graph $\rightarrow$ just remove entries in the tree up to the node where obstacle was removed
- boundaries are fix (don't adapt to the obstacles)



Octree decomposition

- extend to higher dimensions: subdivide volume into 8 subvolumes
- not as optimal, but don't need to be rebalance if information is removed from system

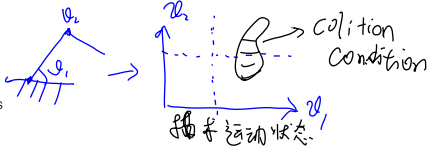
4.3. POTENTIAL FIELD

- mostly used for mobile robots

- Define a potential function over the free space that has a global minimum at the goal and follow the steepest descent of the potential function

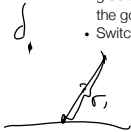
4.3.1. FUNCTION OF THE POTENTIAL FIELD

- potential field: scalar function over the free space
- Navigation: applies a force proportional to the negated gradient of the potential field → combination of repulsive and attractive forces
- Navigation Function is a ideal potential field that
 - has a global minimum at the goal
 - has no local minima
 - grows to infinity near obstacles
 - is smooth
- simplest function would be cone
- high gradient = high velocities, low gradient = low velocities



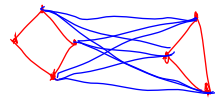
Designing the free space function

- $u(x) = k^*|x|$ → not a good function because velocity will not slow down at the goal and will overshoot
- $u(x) = x^2$ → better function because velocity is proportional to the gradient of the function, no constant gradient, but system will slow down, while approaching the goal ⚡ robot velocity may (very far away from the goal) reach and exceed the maximum velocity of the function due to high slope
- Switch from quadratic to line function once the maximum velocity is reached:



$$U_{att}(q) = \begin{cases} \frac{1}{2}\zeta d^2(q, q_{goal}), & d(q, q_{goal}) \leq d_{goal}^* \\ d_{goal}^* \zeta d(q, q_{goal}) - \frac{1}{2}\zeta (d_{goal}^*)^2, & d(q, q_{goal}) > d_{goal}^* \end{cases}$$

$$\nabla U_{att}(q) = \begin{cases} \zeta (q - q_{goal}), & d(q, q_{goal}) \leq d_{goal}^* \\ d_{goal}^* \zeta, & d(q, q_{goal}) > d_{goal}^* \end{cases}$$



- q_{goal} : position of the goal
- d_{goal} : distance from goal
- d_{goal}^* : threshold distance from the goal where the planner switches between conic and quadratic potentials
- if gradient is 0: minimum, saddle point or maximum → calculate next derivative:
 - < 0 → minimum,
 - > 0 → maximum
- if saddle point: push of the robot would keep it rolling so that it doesn't stop at a step (saddle point)

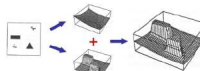
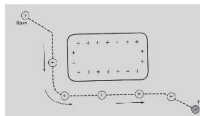
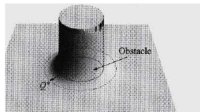
Designing the obstacle function

- Obstacle = repulsive force: keeps robot away from obstacle, the closer the robot is to an obstacle, the stronger the repulsive force should be

$$U_{rep}(q) = \begin{cases} \frac{1}{2}\eta \left(\frac{1}{D(q)} - \frac{1}{Q^*} \right)^2, & D(q) \leq Q^* \\ 0, & D(q) > Q^* \end{cases}$$

$$\nabla U_{rep}(q) = \begin{cases} \eta \left(\frac{1}{Q^*} - \frac{1}{D(q)} \right) \frac{1}{D^2(q)} \nabla D(q), & D(q) \leq Q^* \\ 0, & D(q) > Q^* \end{cases}$$

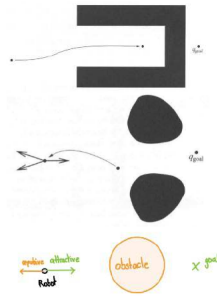
- Q^* : distance from the obstacle that define how large the influence of the obstacle on the robot is
- η :
- $D(q)$: distance to the obstacle
- goes to infinity close to the obstacle



- Resulting force = combination of attractive and repulsive field
- can start with empty map: guided by the force of the goal and the obstacles

Local Minima Problem Ch 4.4

- local minima can appear if a specific form if an obstacle exists
- **U-shape**: the closer the robot comes to the wall, the higher will be the repulsive form $\rightarrow$ attractive = repulsive force $\rightarrow$ stuck in the local minimum
 - even if slightly moved to left and right: robot will stay at the place
 - $\rightarrow$ increase η or Q^* (but robot will not reach goal in the U-shape)
 - solution: run the algorithm and if goal is not reached, increase Q^* and run again
- **Two obstacles**: at same point repulsive forces of the two obstacles = attractive force $\rightarrow$ robot won't continue despite if the obvious path
 - not a saddle point because if we move slightly the robot, will not move further
 - solution: decrease Q^*
- **Saddle point**: no local minima, because slightly push to the right or left will create a small motion so that the robot can go around the obstacle $\rightarrow$ no need to change the parameters but just slightly movement



Summary: Potential field method (Khatib 1986)

- may not find a global solution but can find specific solution if parameters are changed

$$F_{att} = -k_{att} (x - x_{goal})$$

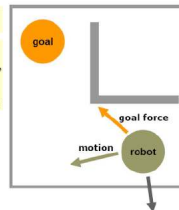
$$F_{rep} = \begin{cases} k_{rep} \left(\frac{1}{\rho} - \frac{1}{\rho_0} \right) \frac{1}{\rho^2} \frac{\partial \rho}{\partial x} & \text{if } \rho \leq \rho_0 \\ 0 & \text{if } \rho > \rho_0 \end{cases}$$

k_{att}, k_{rep} : positive scaling factors

x : position of the robot

ρ : distance to the obstacle

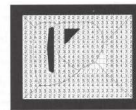
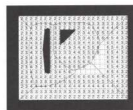
ρ_0 : distance of influence



[Khatib, 1986]

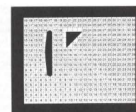
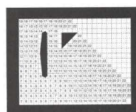
4.3.2. BUSHFIRE ALGORITHM CH 4.3.2

- method to compute distance on a grid (map representation)
- Implementation:
 - all cells near neighbored to obstacles labeled with 2
 - all empty cells neighbored to 2 are labeled with 3
 - iterate as long as there are empty cells
- Four (respects Manhattan distance function) or eight point connectivity
- Gradient: vector which point to the lowest neighbor $\rightarrow$ compute repulsive function



4.3.3. WAVE FRONT PLANNER CH 4.5

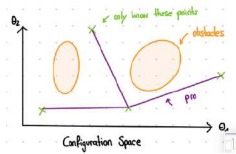
- simple solution to local minima problem
- Implementation:
 - goal labeled with two
 - all neighbors to goal labeled with 3
 - ...
 - determine path via gradient descent start from the start: go to pixel with lower value



5. Sampling-Based Algorithms Ch7

Difficulty with classical approaches:

- Running time increases exponentially with the dimension of the configuration space: For a d-dimension grid with 10 grid points on each dimension, there 10^d grid cells
- Several variants of the path planning problem have been proven to be PSPACE-hard
- not feasible to have complete knowledge about all obstacles in the environment
- Classical approaches not possible in $> 3D$



→ solution: sampling based methods — instead of representing

C_{free} explicitly and globally, we instead "probe" the space locally, as necessary

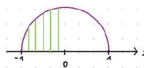
- Sampling-based methods employ a variety of strategies for generating samples (collision-free configurations of the robot) and for connecting the samples with paths to obtain solutions to path-planning problems
- Not sample entire space, but rely on very few samples in the free space

5.1. CHARACTERISTICS OF SAMPLING-BASED PLANNERS

- No attempt to explicitly construct the boundaries of the configuration space obstacles or represent cells of the free space, but rely on whether a configuration is in collision with obstacle or not
- Heuristic algorithm: Unreliable (Example: potential field)
- Give up completeness:
 - Completeness requires that the planner always answers a path-planning query correctly. A complete algorithm finds a path if one exists and reports no otherwise (Example: Canny's roadmap method)
 - Complete planners cannot be implemented for robots with more than 3 degrees of freedom due to their high combinatorial complexity
 - Weaker form of completeness: *probabilistic completeness* → If there is a solution path, the algorithm will find it with high probability

5.2. SAMPLING STRATEGY: CREATE A UNIFORM DISTRIBUTION

- Straight interval
 - Pick a random interval on the unit interval $[0, 1]$ → create unit square/cubes
 - Scale on unit interval: $x_1 = (a - b)x + a$
- Circle:



- Method: Pick x uniform at random from $[-1, 1]$, Set $y = \sqrt{1 - x^2}$ → Intervals of same widths are sampled with equal probabilities

- Method: Pick θ uniformly at random from $[0, 2\pi]$, Set

$x = \cos \theta$ and $y = \sin \theta$ → Circular arcs of same angles are sampled with equal probabilities.



- Both are uniform in some sense. For sampling orientations in 2-D, the second method is usually more appropriate.

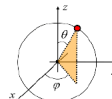
- Sphere:

- Spherical patches of same areas are sampled with equal probabilities.
- Suppose U_1 and U_2 are chosen uniformly at random from $[0, 1]$.

- Unit quaternion:

$$\left(\cos \xi/2, n_x \sin \xi/2, n_y \sin \xi/2, n_z \sin \xi/2 \right) \text{ with } n_x^2 + n_y^2 + n_z^2 = 1$$

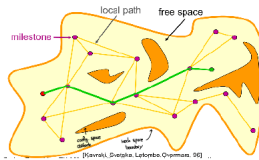
$$\begin{cases} n_x = U_1 \\ n_y = R \cos(2\pi U_2) \\ n_z = R \sin(2\pi U_2) \end{cases} \quad \text{where } R = \sqrt{1 - U_1^2}$$



5.3. CLASSIC MULTIPLE QUERY PRM (PROBABILISTIC ROADMAP)

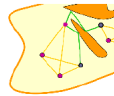
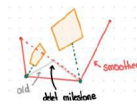
- Probabilistic Roadmaps for Path Planning in High-Dimensional Configuration Spaces, L. Kavraki et al., 1996.
- Assumption: Obstacles are static → Roadmaps don't need to be modified
- Multiple query: No knowledge a priori about start and end point, can be anywhere → Many queries to be processed in the same environment
- Example: Navigation in a static virtual environment, Robot manipulator arms in a work cell

- Planning divided into two phases:
 - learning phase, during which a roadmap in free space is built and is stored as a graph
 - query phase, during which user-defined query configurations are connected with the precomputed roadmap



Implementation of Learning Phase

- Connection Step
 - Set milestones:** Create n uniform distribute milestones: If milestone is in free space $\rightarrow$ keep it, if outside of workspace or in the obstacle $\rightarrow$ try again
 - Find local paths:** If there is no collision along the local path, connect two milestones with straight line doing dense sampling (make sure there is no collision with many tests). This leads to a significant reduction of collision tests to build a roadmap, finding a solution from start to goal via green line (in the image above).
 - Smoothing the resulting paths:** milestone with the maximum distance from line is the new milestone and delete the others if path is collision free.
 - If collision: repeat and use milestone with highest distance to the line
 - $\rightarrow$ fewer velocity changes $\rightarrow$ more efficient BUT homotopic paths needs to be preserved
- Expansion Step (resampling)
 - easy scenes give large set of milestones gives well connected roadmap ⚡ more constrained situations: roadmap consists of several small connected components
 - expand milestones in those regions: If you hit obstacle, continue in random direction (random walk) with max. distance L and then create a new milestone $\rightarrow$ still keeps samples low but helps us to get samples in the problematic areas
 - Could solve problem to get through narrow passages



Uniform sampling

Input: geometry of the moving object & obstacles

Output: roadmap $G = (V, E)$

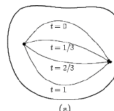
```

V ← ∅ and E ← ∅,
repeat
  q ← a configuration sampled uniformly at random from C.
  if CLEAR(q) then
    Add q to V.
    N, <a set of nodes in V that are close to q.
    for each q' ∈ N, in order of increasing d(q,q')
      if LINK(q',q) then
        Add an edge between q and q' to E.
  
```

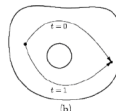
Homotopic paths

- two paths τ and τ' with the same endpoints are homotopic if one can be continuously deformed into the other:

$$h : [0,1] \times [0,1] \rightarrow F \text{ with } h(s,0) = \tau(s) \text{ and } h(s,1) = \tau'$$
- A homotopic class of paths contains all paths that are homotopic to one another
- During the deformation, path should not intersect with any obstacle
- Homotopy ensures that after the path correction, the space topology gets visited from the same side as usually planned. While deforming, no other object may be intersected. It is important for a surveillance robot, which needs to observe e.g., specific walls to check for paintings.



Homotopic example



non-homotopic paths

Query processing in an online step

- During the query phase, paths are to be found between arbitrary input start and goal configurations, using the roadmap constructed in the learning phase.
- Connect q_{init} and q_{goal} to the roadmap
- Start at q_{init} and q_{goal} , Perform a random walk, and try to connect with one of the milestones nearby
- Try multiple times
- If a path is returned, the answer is always correct.
- If no path is found, the answer may or may not be correct. We hope it is correct with high probability.

Why does it work? —> Intuition

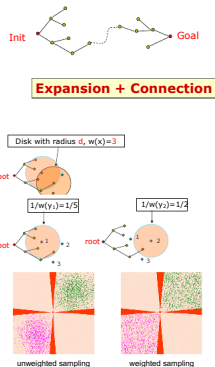
- A small number of milestones almost cover the entire configuration space

Drawbacks of Multi-Query Methods

- PRM is known as „multi-query“ sampling-based method because after initial roadmap is built, multiple queries can be executed on the same roadmap
- lot of useless information stores if we want to run a single query —> developed single-query methods

5.4. SINGLE QUERY PRM CH7.2

- Multiple Query: start and end point anywhere in the free space —> create many paths in the space to find the shortest —> create a roadmap that captures the connectivity of the free space and then answer multiple user-defined queries very fast
- Single Query: is there a possibility to connect start and end point? use the environment only once, eventually not necessary to explore whole space —> don't focus on the exploration of the entire free space, just if its possible to find a path
- Implementation:
 1. Grow two trees from Init position and Goal configurations.
 2. **Expansion:** Randomly sample nodes around existing nodes.
 3. **Connection:** Connect a node in the tree rooted at Init to a node in the tree rooted at the Goal (if distance is below a threshold L and x in Init tree can see y in Goal tree can see)
- ⚡ if nodes are distributed randomly they could not expand but go back —> specify the Expansion
- Specified Expansion:
 1. Pick a node x with probability $1/w(x)$.
 2. Randomly sample k points around x .
 3. For each sample y , calculate $w(y)$, which gives probability $1/w(y)$
 —> high probability for nodes with few nodes in the convergence circle
- Sampling distribution
 - Weight $w(x)$ = no. of neighbors
 - Roughly $\Pr(x) \sim 1 / w(x)$
 - —> place nodes close to the obstacle: possible to go trough narrow passages (right figure)
- Termination Condition: The program iterates between **Expansion** and **Connection**, until
 - two trees are connected —> success
 - max number of expansion & connection steps is reached —> no success

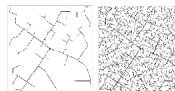


5.4.1. RAPIDLY-EXPLORING RANDOM TREES (RRT, BY LAVALLE & KUFFNER) CH 7.2.2

- single-query planning algorithm that efficiently covers the space between q_{init} and q_{goal} , probabilistically complete, not a roadmap, but a tree

Construction

- grows a tree rooted at the starting configuration by using random samples from the search space. As each sample is drawn, a connection is attempted between it and the nearest state in the tree. If the connection is feasible, this results in the addition of the new state to the tree.
- With uniform sampling of the search space, the probability of expanding an existing state is proportional to the size of its Voronoi region. As the largest

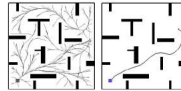


Voronoi regions belong to the states on the frontier of the search, this means that the tree preferentially expands towards large unsearched areas

- Extend the tree from existing nodes in random directions with decreasing distances. Stop edge close to the obstacle
- If obstacle: Stop at the border of the obstacle and move the node to the boundary $\rightarrow$ nodes very close to obstacles, good for narrow passages
- Growth limit Δq : limiting the length of the edge $\rightarrow$ growth limited to this parameter, if no growth limit: explore whole space
- generate nodes close to the goal $\rightarrow$ preference of growing to the goal

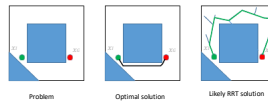
Kinodynamic RRT

- Besides solving single-query problems faster, Kinodynamic RRT is suitable for solving problems for systems with **differential constraints** (e.g. non-holonomic systems such as cars) ⚡ Standard PRM and RRT cannot be applied
- respect the differential constraints $\rightarrow$ Need to compute path more carefully: needs to solve a boundary value problem (differential equations)
- $\rightarrow$ Paths are not lines anymore, but curves (in germany: clothoids (curvature is getting narrower))



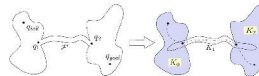
When does PRM and RRT work well? $\rightarrow$ Re-Wiring

- PRM: good for reasonably high-dimensional problems because problems are „easy“ / η , α , β - expansive
- but its also possible to construct instance to make PRM/RRT produce long paths $\rightarrow$ need to keep re-wiring the graph structure: RRT* (Karaman & Frazzoli)
 - for each new sample x_n , check its $\log(n)$ neighborhood
 - if there are better paths from x_1 to x_n , pick that path
 - RRT* is an asymptotically optimal sampling based algorithm $\rightarrow$ As the number of samples goes to infinity, an optimal path from x_1 to x_G



- Problem: Robot traveling between two spaces $\rightarrow$ Many samples wasted with uniform samples because many samples are not part of the free space $\rightarrow$ Extensions: Three possibilities

1. Divide space into subspaces (expansive components)
2. Use geometric transformations to increase the expansiveness of a free space, e.g., smaller variance of uniform samples in narrow passages
3. Integrate the new planner with other planner for multiple-query path planning problems.



5.5. ANALYSIS OF PRM: EXPANSIVE SPACES CH 7.4.2

- Path Planning in Expansive Configuration Spaces, D. Hsu, J.C. Latombe, & R. Motwani, 1999.
- How to determine the number of configurations that should be generated to ensure that, with probability exceeding a given constant, each roadmap is connected
- Issues of PRM: Coverage and Connectivity $\rightarrow$ find a method to find properties and derive the correct number of milestones
- Make sure that

- **coverage is adequate:** It means that milestones are distributed such that almost any point of the configuration space can be connected by a straight line segment to one milestone. $\rightarrow$ uniform distribution
- **milestones are connected:** There should be a one-to-one correspondence between the connected components of the roadmap and those of $F \rightarrow$ not two separated graphs
 - Connectivity is difficult to capture when there are narrow passages ⚡ narrow passages are difficult to define
- Characterize coverage & connectivity? $\rightarrow$ α , β , η - Expansiveness in order to capture the complexity of a configuration space due to narrow passages

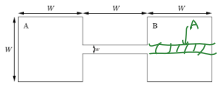


Figure 7.10 An example of an (α, β) -expansive Q_{free} with $\alpha, \beta = w/W$. The points with the smallest α are located in the narrow passage between square A and square B. Each such point sees only a subset of Q_{free} of volume approximately Ww . Hence $\alpha \approx w/W$. A point near the top right corner of square A, sees the entire square, but only a subset of A , of approximate volume αW , contains points that each see a set of volume $2wW$ (hence $\alpha \approx w/W$ and $\beta \approx w/W$). (From Hsu [192].)

$$F = 2W + W \cdot W$$

$$\alpha = \frac{A}{F}$$

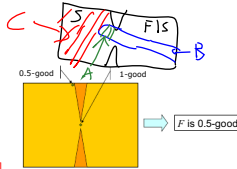
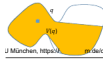
$$S = W \cdot W$$

$$E = \frac{F}{S} = \frac{2W + W \cdot W}{W \cdot W} = \frac{2}{W} + 1$$

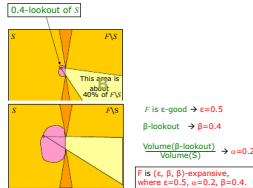


- Definition: Visibility set S of q
- $\text{reach}(S) = \{x \in F \mid \exists y \in S \text{ such that } \overline{xy} \subset F\}$
- All configurations that can be connected to q by a straight line path in F
- All configurations seen by q
- Definition: ϵ -good
 - $\epsilon \leq \frac{S}{F}$
 - $F = C_{\text{free}}$ is η -good if given any point $q \in F$, $\mu(V(q)) \geq \zeta \mu(F)$ (μ measures a volume)
 - Every free configuration sees at least ϵ fraction of the free space, ϵ in $(0,1]$

$$\epsilon = \frac{S}{F} ?$$



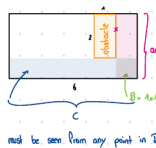
- Definition: lookout of a subset S
 - Subset of points in S that can see at least β fraction of $F \setminus S$, β is in $(0,1]$.
- $\beta = \frac{B}{F \setminus S}$
- $\beta = \frac{A}{S}$
- A and B as big as possible: they will reduce the number of try outs to get a fully connected graph



- Definition (η, α, β) -expansive: The free space F is (η, α, β) -expansive if
 - free space F is η -good
 - for each subset S of F , its β -lookout is at least α fraction of S
- The first condition of definition ensures that a certain fraction of F is visible from any configuration in F . The second condition ensures that each subset $S \subseteq F$ has a large lookout set. It is reasonable to think of S as the union of the reachability sets of a set V of points. Large values of α and β indicate that it is easy to choose random points from S such that adding them to V results in significant expansion of S . This is desirable since it allows for a quick exploration of the entire space.
- bigger η and α and $\beta \rightarrow$ lower cost of constructing a roadmap with good connectivity and coverage

3.2b)

to look at: verify adding for each parameter a small constant how expansiveness of a space can be guaranteed with the (η, α, β)



ϵ : minimum visibility $\rightarrow$ pick point that can see the smallest area of the free space

$$\epsilon = \frac{A}{F} = \frac{2}{3 \cdot 4 - 2 \cdot 4} = \frac{2}{12 - 8} = \frac{2}{4}$$

α : region in confined space which allows me to look in the free space

$$\text{lookout region } \alpha = \frac{B}{F \setminus S} = \frac{4}{8}$$

β : ratio of region that can connect to visible space where I'm connecting

$$\beta = \frac{S}{F \setminus S} = \frac{4}{12 - 8} = \frac{4}{4}$$

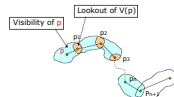
$$\frac{8 \ln(8/\epsilon \beta \beta)}{\epsilon \beta} + \frac{3}{\beta}$$

- Theorem: A roadmap of $n = \frac{8 \ln(8/\epsilon \beta \beta)}{\epsilon \beta} + \frac{3}{\beta}$ uniformly-sampled milestones has the correct connectivity with probability at least $1 - \gamma$ (gamma: failure rate $\rightarrow$ exponential dependence of success) (each subgraph is a connected graph)

- Definition: Linking sequence

17

$$\epsilon \alpha (n - \frac{3}{\beta}) = \frac{6}{\epsilon \alpha \beta}$$



1 increase n based on (η)

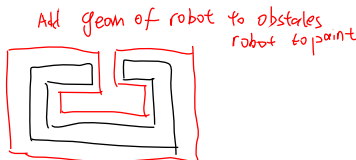
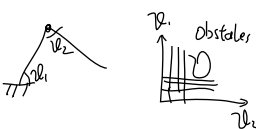
$$\frac{1}{\gamma} = n$$

$$(1 - \gamma) = k_1 e^{-k_2 n}$$

2 random walk

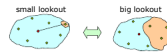
$$G = T(\sqrt{2})$$

exercise



Add geom of robot to obstacles
robot to point

- p_{n+1} is chosen from the lookout of the subset seen by $p, p_1, \dots, p_n$
- place a point p_1 in the lookout region of the parent point $\rightarrow$ defines how much of the volume is visible
- C-Space with a larger lookout set has a higher probability of constructing a linking sequence
- Lemma: In an expansive space with large η, α , and β , we can obtain a linking sequence that covers a large fraction of the free space, with high probability.



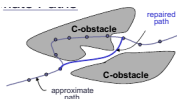
- **Theorem 1:** Probability of achieving good connectivity increases exponentially with the number of milestones (in an expansive space).
- If (η, α, β) decreases $\rightarrow$ then need to increase the number of milestones (to maintain good connectivity)
- **Theorem 2:** Probability of achieving good coverage, increases exponentially with the number of milestones (in an expansive space).
- Probabilistic completeness: In an expansive space, the probability that a PRM planner fails to find a path when one exists goes to 0 exponentially in the number of milestones ($\sim$ running time).

Summary

- Main result: If a C-space is expansive, then a roadmap can be constructed efficiently with good connectivity and coverage.
- Limitation in practice: It does not tell you when to stop growing the roadmap. A planner stops when either a path is found or max steps are reached.

5.6. PRM CONCLUSION

- A relatively small number of milestones and local paths are sufficient to capture the connectivity of the free space $\rightarrow$ Exponential convergence in expansive free space (probabilistic completeness)
- Checking sampled configurations and connections between samples for collision can be done efficiently $\rightarrow$ Hierarchical collision checking
- Problem: Even with the best sampling methods, roadmaps may not contain valid solution paths
 - may lack critical cdfs in narrow passages
 - may contain approximate paths that are 'nearly' valid
- Repairing/Improving Approximate Paths
 - Create initial roadmap Extract *approximate path* P
 - Repair P (push to C-free)
 - Focus search around P
 - Use OBPRM-like techniques

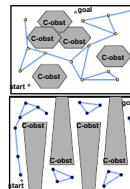


Pros:

- PRMs are *probabilistically complete*
- PRMs apply easily to high-dimensional C-space
- PRMs support fast queries w/ enough preprocessing
- Many success stories where PRMs solve previously unsolved problems

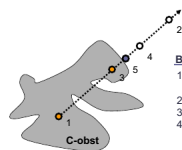
Cons:

- PRMs don't work as well for some problems: unlikely to sample nodes in *narrow passages*, hard to sample/connect nodes on constraint surfaces
- Our work concentrates on improving PRM performance for such problems.



5.7. OBPRM: OBSTACLE-BASED PRM

- To Navigate Narrow Passages we must sample in them: most PRM nodes are where planning is easy (not needed)
- Idea: Can we sample nodes near C-obstacle surfaces?
 - we cannot explicitly construct the C-obstacles but we do have models of the (workspace) obstacles
- 1. Find a point in S's C-obstacle (robot placement colliding with S)
- 2. Select a random direction in C-space
- 3. Find a free point in that direction
- 4. Find boundary point between them using binary search (collision checks)



- Note: we can use more sophisticated heuristics to try to cover C-obstacle
- Not for dynamic scenes

6. Kalman Filtering Ch 8

In this chapter, we begin to consider cases for which the robot's knowledge of the world derives from measurements provided by imperfect, noisy sensors.

Improve the knowledge about the world by moving the robot in the environment.

Based on G. Welch & G. Bishop, "An Introduction to the Kalman Filter"

Why Kalman Filtering? Kalman Filter is a tool that predicts values using a bunch of mathematical equations under the assumptions that input data is in the form of Gaussian distribution and then followed by the application of linear equations to that Gaussian distribution. The filter is very powerful in several aspects: it supports estimations of past, present, and even future states, and it can do so even when the precise nature of the modeled system is unknown.

Kalman filter (KF) techniques have been successfully used to predict and estimate errors in position of linear systems like ships.

What should we do if the sensors do not agree on the measurement? The mean of the Gaussian with smaller sigma should be trusted more. Weight the mean according to sigma and combine the measurements.

Idea of the Kalman Filter:

- The main idea behind the Kalman filter is that you do not just have an estimate for a parameter x but also have some estimate for the uncertainty in your value for x
- This is represented by the variance/covariance of the estimate P_x
- There are many advantages to this, as it allows you a means for estimating the confidence in your robot's ability to execute a task (e.g. navigating through a tight doorway)
- In the case of the KF, it also provides a nice mechanism for optimally combining data over time
- This optimality condition assumes we have **linear models**, and the error characteristics of our sensors can be modeled as **zero-mean, Gaussian noise**

6.1. DATA FUSION EXAMPLE

- Use code from Team 1 and Team 2 to obtain two different measurements $Z = [z_1, z_2]^T$ for the range r to a beacon
- Variance in each of these sensor measurements is R_1 and R_2 , respectively
- Q: How should we fuse these measurements in order to obtain the "best" possible resulting estimate for r ?
- We'll define "best" from a least-squares perspective...
- We have 2 measurements that are equal to r plus some additive zero-mean Gaussian noise v_1 and v_2

$$z_1 = r + N(0, R_1) = r + v_1$$

$$z_1 = r + N(0, R_1) = r + v_1$$

- We want to fuse these measurements to obtain a new estimate for the range $\hat{r}$
- Using a weighted least-squares approach, the resulting sum

$$\text{of squares error will be } e = \sum_{i=1}^n w_i (\hat{r} - z_i)^2$$

- Minimizing the error: $\frac{\partial e}{\partial \hat{r}} = \frac{\partial}{\partial \hat{r}} \sum_{i=1}^n w_i (\hat{r} - z_i)^2 = 2 \sum_{i=1}^n w_i (\hat{r} - z_i) = 0 \Rightarrow \hat{r} = \frac{\sum_{i=1}^n w_i z_i}{\sum_{i=1}^n w_i}$

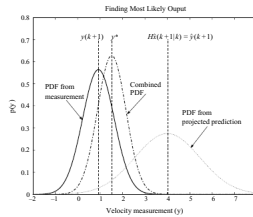


Figure 8.6 Measurements and predictions are then merged. The PDF plotted with the dashed line results from the combination of the measurement PDF and the PDF of the prediction projected into output space, where the combination is computed using theorem 8.2.1. The most likely output y^* is the value at which this combined distribution reaches its peak.

- We obtain: $\hat{r} = \frac{z_1}{\frac{1}{R_1} + \frac{1}{R_2}} = \frac{R_2}{R_1 + R_2} z_1 + \frac{R_1}{R_1 + R_2} z_2$ with $w_i = \frac{1}{\sigma_i^2} = \frac{1}{R_i}$
- Rewrite as **Kalman equation**: $\hat{r} = z_1 + \frac{R_1}{R_1 + R_2} (z_2 - z_1)$ with the **Kalman gain** $K = [0, 1]$

- $R_i \rightarrow \infty$: rely on second estimate
- first set $\hat{r}$ to z_1 and the shift to z_2 according to the R values or if we think of this as adding a new measurement (z, P) to our current estimate ($\hat{r}, R$) of the state we would get ($\rightarrow$ prediction):

$$\hat{r}_{k+1} = \hat{r}_{k+1}^- + \frac{P_{k+1}^-}{P_{k+1}^- + R} (z_{k+1} - \hat{r}_{k+1}^-) = \hat{r}_{k+1}^- + K_{k+1} (z_{k+1} - \hat{r}_{k+1}^-)$$

- For merging Gaussian distributions, the update rule is: $\frac{1}{\sigma_3^2} = \frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2} = \frac{\sigma_1^2 + \sigma_2^2}{\sigma_1^2 \sigma_2^2} \Rightarrow \sigma_3^2 = \frac{\sigma_1^2 \sigma_2^2}{\sigma_1^2 + \sigma_2^2}$
- which if we write in our measurement update equation form we get:

$$P_{k+1} = \frac{P_{k+1}^- R_{k+1}}{P_{k+1}^- + R_{k+1}} \equiv P_{k+1}^- - K_{k+1} P_{k+1}^-$$

Time Update

- Step 1** of the time update phase is only our prediction based upon the linear state update equation that we have $\bar{x}_{k+1}^- = A \bar{x}_k + B \bar{u}_k$
- A: state transition matrix, x_k : current state (either position, position + velocity or position + velocity + acceleration) $\rightarrow$ estimate v and a
- B: process parameter u_k from robot controller $\rightarrow$ know v and $a \rightarrow$ better prediction if we have access to the control values
- Step 2** of the time update phase comes from projecting our covariance matrix forward where we only add the process noise variance Q due to the normal sum distribution property where $\sigma_3^2 = \sigma_1^2 + \sigma_2^2$

Example: $\hat{u}_k$

x includes position and velocity ($\hat{v}$)

position $\hat{p}$

$\hat{v}$ and $\hat{a}$ from robot controller $\hat{u}_k = \begin{pmatrix} \hat{v} \\ \hat{a} \end{pmatrix}$

$\bar{x}_{k+1}^- = A \bar{x}_k + B \bar{u}_k$

$\begin{pmatrix} \hat{p} \\ \hat{v} \end{pmatrix} = \begin{pmatrix} 1 & \Delta t \\ 0 & 1 \end{pmatrix} \begin{pmatrix} \hat{p} \\ \hat{v} \end{pmatrix} + \begin{pmatrix} \Delta t \\ 1 \end{pmatrix} \begin{pmatrix} \hat{v} \\ \hat{a} \end{pmatrix}$

prediction $\hat{v}$ and $\hat{a}$ from robot controller if v is constant

Example: $\hat{u}_k$

x includes only position, but we know v and a from robot controller $\hat{u}_k = \begin{pmatrix} \hat{p} \\ \hat{v} \end{pmatrix}$

$\bar{x}_{k+1}^- = A \bar{x}_k + B \bar{u}_k$

$\hat{p}_{k+1} = \hat{p}_k + \Delta t \hat{v}_k + \frac{\Delta t^2}{2} \hat{a}_k$

$\hat{v}_{k+1} = \hat{v}_k + \Delta t \hat{a}_k$

$\hat{a}_{k+1} = \hat{a}_k$

$\hat{u}_{k+1} = \begin{pmatrix} \hat{p}_{k+1} \\ \hat{v}_{k+1} \\ \hat{a}_{k+1} \end{pmatrix}$

$\hat{u}_k = \begin{pmatrix} \hat{p}_k \\ \hat{v}_k \\ \hat{a}_k \end{pmatrix}$

$\hat{u}_{k+1} = \begin{pmatrix} \hat{p}_{k+1} \\ \hat{v}_{k+1} \\ \hat{a}_{k+1} \end{pmatrix}$

$\hat{u}_k = \begin{pmatrix} \hat{p}_k \\ \hat{v}_k \\ \hat{a}_k \end{pmatrix}$

$\hat{u}_{k+1} = \begin{pmatrix} \hat{p}_{k+1} \\ \hat{v}_{k+1} \\ \hat{a}_{k+1} \end{pmatrix}$

$\hat{u}_k = \begin{pmatrix} \hat{p}_k \\ \hat{v}_k \\ \hat{a}_k \end{pmatrix}$

$\hat{u}_{k+1} = \begin{pmatrix} \hat{p}_{k+1} \\ \hat{v}_{k+1} \\ \hat{a}_{k+1} \end{pmatrix}$

$$\text{Covariance matrix: } P_k = \frac{1}{N} \sum_{i=1}^N (\bar{x}_i - \bar{\mu}_k) (\bar{x}_i - \bar{\mu}_k)^T$$

$$\text{Prediction of next covariance (model error } Q): P_{k+1} = \frac{1}{N} \sum_{i=1}^N (\bar{x}_{i,k+1} - \bar{\mu}_{k+1}) (\bar{x}_{i,k+1} - \bar{\mu}_{k+1})^T + Q$$

- No control value ($u_k=0$), replace x_{k+1} :

$$P_{k+1} = \frac{1}{N} \sum_{i=1}^N \left[A (\bar{x}_{i,k}^* - \bar{\mu}_k) \right] \left[A (\bar{x}_{i,k}^* - \bar{\mu}_k) \right]^T + Q = A P_k A^T + Q$$

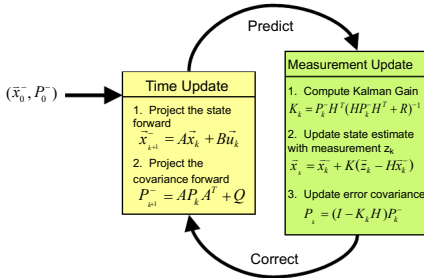
- Q will flatten the current prediction (flatter gaussian)

- H: measurement matrix (e.g. x is a position (x, y) and z is rotated version (x', y') $\rightarrow$ constant linear error corrected by measurement error, here: rotation matrix)
- add measurement z_k , otherwise the distribution will flatten

6.2. OBSERVABILITY MATRIX

- A system with state vector x of dimension n is observable if the observability matrix has row rank n (i.e. n linearly independent rows)
- Example:

$$\begin{aligned} & A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} & & & C = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \\ & \text{Measurement: position } p & & & \text{Measurement: speed } v \\ & H = \begin{pmatrix} 1 & 0 \end{pmatrix} & & & H = \begin{pmatrix} 0 & 1 \end{pmatrix} \\ & O = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} & & & O = \begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix} \end{aligned} \quad \text{and} \quad \text{O} = \begin{bmatrix} H \\ HA \\ \vdots \\ HA^{n-1} \end{bmatrix}$$



- Smaller $Q \rightarrow$ Trust chosen model
- Smaller $R \rightarrow$ Trust measurements

6.3. PROBLEMS WITH THE KALMAN FILTER

- For many applications, the time update and measurement equations are NOT linear
- As a consequence, the KF is not applicable
- However, the KF is such a nice algorithm that maybe if we linearize around the non-linearities, we can still get good performance in practice
- This line of thought lead to the development of the Extended Kalman Filter (EKF)
- By relaxing the linear assumptions, the use of the KF is extended dramatically
- Life Rule: There is no such thing as a free lunch
- We can no longer use the word "optimal" with the EKF

6.4. KALMAN EXAMPLES

- The Kalmanfilter is a very powerful tool when it comes to controlling noisy systems. The basic idea of a Kalman filter is: Noisy data in $\rightarrow$ hopefully less noisy data out
- The applications of a Kalman filter are numerous:
 - Tracking objects (e.g., missiles, faces, heads, hands)
 - Fitting Bezier patches to (noisy, moving, ...) point data
 - Economics
 - Navigation
- Many computer vision applications: Stabilizing depth measurements, Feature tracking, Cluster tracking
- – Fusing data from radar, laser scanner and stereo-cameras for depth and velocity measurements

Predict:

Predict-Update Equations

- $\hat{x}$: Estimated state.
- F : State transition matrix (i.e., transition between states).
- u : Control variables.
- B : Control matrix (i.e., mapping control to state variables)
- P : State variance matrix (i.e., error of estimation).
- Q : Process variance matrix (i.e., error due to process).
- y : Measurement variables.
- H : Measurement matrix (i.e., mapping measurements onto state).
- K : Kalman gain.
- R : Measurement variance matrix (i.e., error from measurements).
- Subscripts are as follows: $t|t$ current time period, $t-1|t-1$ previous time period, and $t|t-1$ are intermediate steps.

Update:

$$\begin{aligned}\hat{x}_{t|t-1} &= F_t \hat{x}_{t-1|t-1} + B_t u_t \\ P_{t|t-1} &= F_t P_{t-1|t-1} F_t^T + Q_t \\ \hat{x}_{t|t} &= \hat{x}_{t|t-1} + K_t (y_t - H_t \hat{x}_{t|t-1}) \\ K_t &= P_{t|t-1} H_t^T (H_t P_{t|t-1} H_t^T + R_t)^{-1} \\ P_{t|t} &= (I - K_t H_t) P_{t|t-1}\end{aligned}$$

Possible values of K

- since $K_k = P_k^- H^T (H P_k^- H^T + R)^{-1}$, there are two extreme values K can take

Case 1: $K \rightarrow 0$	Case 2: $K \rightarrow H^{-1}$
• happens if $R \rightarrow \infty$ (sensor is lost/crappy sensor) or $P_k = 0$ (the state estimation is 100% accurate) • The prior estimation is assumed to be exact, therefore the measurement is neglected • KF will blend the resulting mean closer to predicted state mean • Update of the state estimate: $\hat{x}_k = \hat{x}_k$ • Update of the error covariance: $P_k = P_k^- = 0$	• happens if $R = 0$ (the measurement is 100% correct) or $P_k \rightarrow \infty$ (prediction is not valuable at all) • The measurement is fully trusted and the prior estimation is dropped. • KF will blend the mean closer to mean measurements. • Update of the state estimate: $\hat{x}_k = H^{-1} z_k$ • Update of the error covariance: $P_k = 0$

Model Definition Process

The Kalman filter removes noise by assuming a pre-defined model of a system. Therefore, the Kalman filter model must be meaningful. It should be defined as follows:

1. **Understand the situation:** Look at the problem. Break it down to the mathematical basics. If you don't do this, you may end up doing unneeded work.
2. **Model the state process:** Start with a basic model. It may not work effectively at first, but this can be refined later.
3. **Model the measurement process:** Analyze how you are going to measure the process. The measurement space may not be in the same space as the state (e.g., using an electrical diode to measure weight, an electrical reading does not easily translate to a weight).
4. **Model the noise:** This needs to be done for both the **state** diode to measure weight, an electrical reading does not easily translate to a weight).
4. **Model the noise:** This needs to be done for both the **state** and **measurement** process. The base Kalman filter assumes Gaussian (white) noise, so make the variance and covariance (error) meaningful (i.e., make sure that the error you model is suitable for the situation).
5. **Test the filter:** Often overlooked, use synthetic data if necessary (e.g., if the process is not safe to test on a live environment). See if the filter is behaving as it should.
6. **Refine filter:** Try to change the noise parameters (filter), as this is the easiest to change. If necessary go back further, you may need to rethink the situation.

6.4.1. EXAMPLE: WATER LEVEL IN TANK

see [Subject MI63: Kalman Filter Tank Filling](#)

6.5. THE EXTENDED KALMAN FILTER (EKF) CH 8.3

- The Extended Kalman (EKF) is a sub-optimal extension of the original KF algorithm
- The EKF allows for estimation of non-linear processes or measurement relationships
- This is accomplished by linearizing the current mean and covariance estimates (similar to a first order Taylor series approximation)
- Replace predictions function by non-linear functions

EKF Time Update Phase

- However when we propagate the covariance ahead in time, the underlying function needs to be linear in order to properly combine the Gaussian uncertainty in our state x

- Solution: Use a Taylor series

- Let's say we know the uncertainty of a variable x , and we want to compute the uncertainty of $y=f(x)$

- We can then use the Jacobian to linearly approximate y

- So the covariance is projected ahead as $\rightarrow$

- where A is the Jacobian of f with respect to x (state)

and W is the Jacobian of f with respect to w (sensor measurements: velocity)

$$P_{k+1}^- = A P_k^- A^T + Q$$

Kalman Filter

$$P_{k+1}^- = A P_k^- A^T + W Q W^T$$

Extended Kalman Filter

EKF Robot Implementation Example

- Assume that we have a mobile robot using odometry and range measurements to landmark to estimate its position and orientation: $\vec{x} = [x, y, \theta]^T$
- Assume that the odometry provides a velocity estimate V and an angular velocity estimate ω that are both corrupted by gaussian noise
- We can write the state update equation as, which is obviously non-linear in the state

$$\begin{bmatrix} x \\ y \\ \theta \end{bmatrix}_{k+1} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}_k + \begin{bmatrix} V \cos \theta \Delta t \\ V \sin \theta \Delta t \\ \omega \Delta t \end{bmatrix}$$

- Calculate the Jacobian A and W

$$f(\vec{x}) = \begin{bmatrix} f_1(\vec{x}) \\ f_2(\vec{x}) \\ f_3(\vec{x}) \end{bmatrix} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}_{k+1} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}_k + \begin{bmatrix} V \cos \theta \Delta t \\ V \sin \theta \Delta t \\ \omega \Delta t \end{bmatrix} \quad W = \frac{\partial f}{\partial \vec{x}} = \begin{bmatrix} \Delta t \cos \theta & 0 \\ \Delta t \sin \theta & 0 \\ 0 & \Delta t \end{bmatrix} \quad A = \frac{\partial f}{\partial \vec{x}} = \begin{bmatrix} 1 & 0 & -V \sin \theta \Delta t \\ 0 & 1 & V \cos \theta \Delta t \\ 0 & 0 & 1 \end{bmatrix}$$

EKF Measurement Update Phase

- Again, in the measurement update we can have a non-linear relationship between our measurements z_k and state x_k : $\vec{z}_k = h(\vec{x}_k, \vec{v}_k)$ and once again we will assume that the noise is zero $\vec{z}_k = h(\vec{x}_k, 0)$
- To propagate uncertainty, we shall again have to calculate the appropriate Jacobians
- H is the Jacobian matrix of partial derivatives of h with respect to x
- V is the Jacobian matrix of partial derivatives of h with respect to measurement noise v
- These are then substituted into the original KF as appropriate

KF

EKF

- Computing the Kalman Gain:

$$K_i = P_i^* H^T (H P_i^* H^T + R)^{-1}$$

$$K_i = P_i^* H^T (H P_i^* H^T + R)^{-1}$$

- State Update:

$$\vec{x}_i = \vec{x}_i + K_i (\vec{z}_i - h(\vec{x}_i, 0))$$

$$\vec{x}_i = \vec{x}_i + K_i (\vec{z}_i - h(\vec{x}_i, 0))$$

- Covariance Update:

$$P_i = (I - K_i H) P_i^*$$

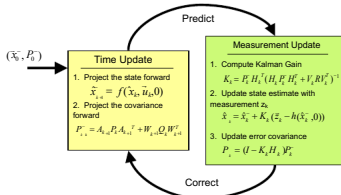
$$P_i = (I - K_i H) P_i^*$$

EKF Robot Implementation Example

- Let's go back to our beacon example for the Albo. From this, our range measurement can be written as $z_k = \sqrt{(x_k - x_b)^2 + (y_k - y_b)^2}$, where x_b and y_b are constants
- The Jacobians H and V can then be calculated as

$$H = \frac{\partial z}{\partial \vec{x}} = \begin{bmatrix} \frac{x_k}{\sqrt{(x_k - x_b)^2 + (y_k - y_b)^2}} & \frac{y_k}{\sqrt{(x_k - x_b)^2 + (y_k - y_b)^2}} & 0 \end{bmatrix} \text{ and } V = 1 \text{ (just linear addition of noise)}$$

- Note that if this were the only measurements available then θ would be unobservable

The discrete extended Kalman Filtercontrol input u is neglected here**Time Update:**

$$\vec{x}_{t+1} = f(\vec{x}_t, 0)$$

$$P_{t+1}^* = F_t P_t F_t^T + G_t Q G_t^T$$

Measurement Update:

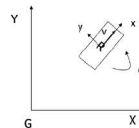
$$K_t = P_{t+1}^* H_t^T (H_t P_{t+1}^* H_t^T + R_t)^{-1}$$

$$\vec{x}_t = \vec{x}_{t+1} + K_t (z_t - h(\vec{x}_{t+1}, 0))$$

$$P_t = P_{t+1}^* - K_t H_t P_{t+1}^*$$

K_t is Kalman Gain, Q is the variance process noise, R is the variance of the measurement noise

$$F_t = \frac{\partial f(\vec{x}_t)}{\partial \vec{x}_t} \Big|_{\vec{x}_t = \vec{x}_{t+1}} \quad H_t = \frac{\partial h(\vec{x}_t)}{\partial \vec{x}_t} \Big|_{\vec{x}_t = \vec{x}_{t+1}} \\ G_t = \frac{\partial g(\vec{x}_t)}{\partial \vec{x}_t} \Big|_{\vec{x}_t = \vec{x}_t} \quad U_t = \frac{\partial u(\vec{x}_t)}{\partial \vec{x}_t} \Big|_{\vec{x}_t = \vec{x}_t}$$



6.6. LINEARIZED MOTION MODEL FOR A ROBOT

- EKF is well suited to the problem of localizing a mobile robot equipped with sensors that can detect range and bearing to previously mapped landmarks in the environment
- Velocities of the robot from a global perspective:
 - $\dot{x}_t = V_t \cos \phi_t$
 - $\dot{y}_t = V_t \sin \phi_t$
 - $\dot{\phi}_t = \omega_t$
- Discrete time state estimate (including noise):
 - $\hat{x}_{t+1} = \hat{x}_t + (V_t + w_{V_t}) \epsilon t \cos \hat{\phi}_t$
 - $\hat{y}_{t+1} = \hat{y}_t + (V_t + w_{V_t}) \epsilon t \sin \hat{\phi}_t$
 - $\hat{\phi}_{t+1} = \hat{\phi}_t + (\omega_t + w_{\omega_t}) \epsilon t$
- The indirect Kalman filter derives the pose equations from the estimated error of the state:
 - $x_{t+1} - \hat{x}_{t+1} = \tilde{x}_{t+1}$
 - $y_{t+1} - \hat{y}_{t+1} = \tilde{y}_{t+1}$
 - $\phi_{t+1} - \hat{\phi}_{t+1} = \tilde{\phi}_{t+1}$
- From the error-state propagation equation, we can obtain the State propagation and noise input functions A and W : $\tilde{X}_{t+1} = F_t \tilde{X}_t + G_t W_t$

$$\begin{bmatrix} \tilde{x}_{t+1} \\ \tilde{y}_{t+1} \\ \tilde{\phi}_{t+1} \end{bmatrix} = \begin{bmatrix} 1 & 0 & -V_m \epsilon t \sin \hat{\phi} \\ 0 & 1 & V_m \epsilon t \cos \hat{\phi} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tilde{x}_t \\ \tilde{y}_t \\ \tilde{\phi}_t \end{bmatrix} + \begin{bmatrix} -\epsilon t \cos \phi_R & 0 \\ -\epsilon t \sin \phi_R & 0 \\ 0 & -\epsilon t \end{bmatrix} \begin{bmatrix} w_{V_t} \\ w_{\omega_t} \end{bmatrix}$$
- compute the standard covariance propagation equation: $P_{t+1/t} = F_t P_{t/t} F_t^T + G_t Q_t G_t^T$

6.7. EXTENDED KALMAN FILTER FOR SLAM CH 8.4

One common approach to solving the SLAM (simultaneous localization and mapping) problem is to use a Kalman filter to simultaneously estimate the position of a moving vehicle along with the positions of landmarks seen by the vehicle.

- Typical problem: Unknown map for the robot $\rightarrow$ Robot needs to build the map while moving in the environment
- State vector: location of the vehicle together with the locations of each of the landmarks, State vector can be grown as new landmarks are discovered
- Position of the robot $\vec{X}_R = [x, y, \theta]^T$
- State vector $\vec{X}_k = [\vec{X}_R \ \vec{X}_{L_1} \dots \vec{X}_{L_n}]^T$, where n is the number of Landmarks

Covariance matrix: $P = \begin{bmatrix} P_{RR} & P_{RL_1} & \dots & P_{RL_N} \\ P_{L_1R} & P_{L_1L_1} & \dots & P_{L_1L_N} \\ \vdots & \vdots & \ddots & \vdots \\ P_{L_NR} & P_{L_NL_1} & \dots & P_{L_NL_N} \end{bmatrix}$ where P_{RR} is the uncertainty from the robot to itself, P_{RL_1}

the uncertainty from the robot to the first landmark, ...

- Kinematic equations for landmark propagation

- Prediction of the robot motion:

$$l\hat{x}_{t+1} = \hat{x}_t + (V_t + w_{V_t}) \delta t \cos \hat{\phi}_t$$

$$\hat{y}_{t+1} = \hat{y}_t + (V_t + w_{V_t}) \delta t \sin \hat{\phi}_t$$

$$\hat{\phi}_{t+1} = \hat{\phi}_t + (\omega_t + w_{\omega_t}) \delta t$$

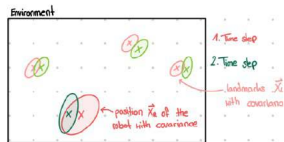
- Prediction of the landmarks will be constant (no change over time):

$$l\hat{x}_{L,t+1} = \hat{x}_{L,t}$$

$$\hat{y}_{L,t+1} = \hat{y}_{L,t}$$

$$\hat{\phi}_{L,t+1} = \hat{\phi}_{L,t}$$

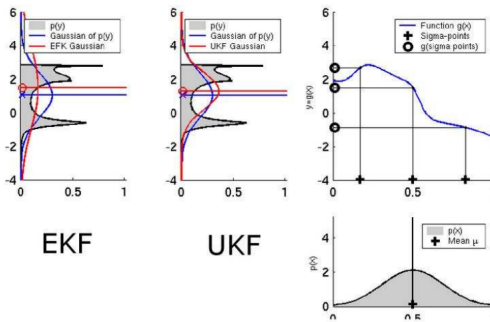
- Sensor equations for update: $\tilde{X} = [\tilde{X}_R^T \ \tilde{X}_{L_1}^T \ \dots \ \tilde{X}_{L_n}^T]^T$
 $H = [H_R \ 0 \ \dots \ 0 \ H_{L_1} \ 0 \ \dots \ 0]$



- Modify entries in H to 0 for landmarks I don't see (here only landmark i can be seen)
- Very powerful because covariance update records *shared information* between landmarks and robot positions
- Solve Data Association step $\rightarrow$ which entries are currently visible and which not
- Intuition: chose data close to the predicted position: $v_{ij} = y_i - h_i(x(k+1 | k), j)$
- Uncertainties need to be considered in the choice of data: $\chi_{ij}^2 = v_{ij}^T S_{ij}^{-1} v_{ij}$, where $S_{ij} = H_i(k+1, j)P(k+1 | k)H_i(k+1, j)^T + R_i(k+1)$
- Mahanalobis norm
 - for a group of values with mean: $\mu = (\mu_1, \mu_2, \mu_3, \dots, \mu_p)$ and a multivariate vector: $x = (x_1, x_2, x_3, \dots, x_p)$
- $D_M(x) = \sqrt{(x - \mu)^T \Sigma^{-1} (x - \mu)}$
- If covariance matrix equals identity matrix then Euclidean distance. If covariance matrix is diagonal then normalized Euclidean distance

6.8. SUMMARY

- For linear processes with Gaussian noise, the KF provides a provably optimal means for fusing measurement information
- For our purposes, the plain KF is too restrictive as both our motion and measurement equations are highly non-linear
- The linear constraints can be lifted, but the optimality of the filter is lost (although performance is still quite good in practice)
- This is the basis for the Extended Kalman Filter (EKF) – one of the most used estimation algorithms in mobile robotics

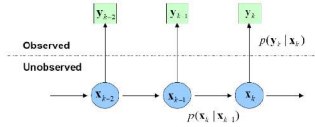


7. Unscented Kalman Filter

In cases the non-linearity too big, then even EKF will not converge. This chapter's filters can handle non-linearities.

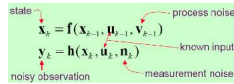
7.1. FILTERING PROBLEM: GENERAL PROBLEM STATEMENT

- Filtering is the problem of sequentially estimating the states (parameters or hidden variables) of a system as a set of observations become available on-line, where $\mathbf{x}_i$ is the state and $\mathbf{y}_i$ the observation
- $\hat{\mathbf{z}} = \hat{\mathbf{H}} \hat{\mathbf{x}} \quad \hat{\mathbf{z}} = \hat{\mathbf{h}}(\hat{\mathbf{x}})$
- Solution of sequential estimation problem given by
 - Posterior Density: $p(\mathbf{X}_k | \mathbf{Y}_k)$ with $\mathbf{X}_k = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{k-1}, \mathbf{x}_k\}$ and $\mathbf{Y}_k = \{\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_{k-1}, \mathbf{y}_k\}$
 - Marginal of the Posterior: $p(\mathbf{x}_k | \mathbf{Y}_k)$
 - One does not need to keep track of the history of the state sequence



- Dynamic State Space Model
 - General discrete time non-linear, non-Gaussian dynamic system
 - Assumption: States are markovian, i.e.,

$$p(\mathbf{x}_k | \mathbf{x}_{k-1}, \mathbf{x}_{k-2}, \dots, \mathbf{x}_0) = p(\mathbf{x}_k | \mathbf{x}_{k-1})$$



- Solution by Extended Kalman Filter $\rightarrow$ Crude Approximation:
 - EKF uses first order terms of the Taylor series expansion of the nonlinear functions.
 - Large errors are introduced when the models are highly non-linear
 - Local linearity assumption breaks down when the higher order terms become significant.

7.2. UNSCENTED TRANSFORMATION

The unscented transformation (UT) is a method for calculating the statistics of a random variable which undergoes a nonlinear transformation and builds on the principle that it is easier to approximate a probability distribution than an arbitrary nonlinear function.

- Problem: Propagating an n_x dimensional random variable $\mathbf{x}$ through a nonlinear function, $g: \mathbb{R}^{n_x} \rightarrow \mathbb{R}^{n_y}$, $\mathbf{y} = g(\mathbf{x})$
- Formulation: Gaussian peak is shifted slightly. Sample the new distribution by choosing sample points and projecting the sampling points (Sigma points) through the function g . Assume $\mathbf{x}$ has mean $\bar{\mathbf{x}}$ and covariance P_x . A set of $2n_x + 1$ weighted samples or sigma points $S_i = \{\mathbf{W}_i, \mathbf{x}_i\}$ are chosen as follows:

$$\begin{aligned} \mathbf{x}_0 &= \bar{\mathbf{x}} \text{ (center)} & \mathbf{W}_0 &= \theta / (n_x + \theta) & i &= 0 \\ \mathbf{x}_i &= \bar{\mathbf{x}} + \left(\sqrt{(n_x + \theta) P_x} \right)_i & \mathbf{W}_i &= 1 / \left\{ 2 (n_x + \theta) \right\} & i &= 1, \dots, n_x \\ \mathbf{x}_i &= \bar{\mathbf{x}} - \left(\sqrt{(n_x + \theta) P_x} \right)_i & \mathbf{W}_i &= 1 / \left\{ 2 (n_x + \theta) \right\} & i &= n_x + 1, \dots, 2n_x \end{aligned}$$

- where k is a scaling parameter and $\left(\sqrt{(n_x + \theta) P_x} \right)_i$ is the i -th row or column of the matrix square root of

$$(n_x + \theta) P_x, \mathbf{W}_i \text{ is the weight associated with the } i\text{-th point such that } \sum_{i=0}^{2n_x} \mathbf{W}_i = 1.$$

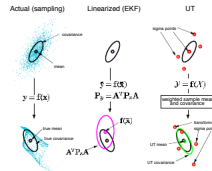
- Each sigma point is propagated through the nonlinear function $\mathbf{Y}_i = g(\mathbf{x}_i) \quad i = 0, \dots, 2n_x$ and the estimated mean and covariance are computed as follows

$$\bar{y} = \sum_{i=0}^{2n} W_i Y_i \quad P_y = \sum_{i=0}^{2n} W_i (Y_i - \bar{y}) (Y_i - \bar{y})^T$$

- These estimates of the mean and covariance are accurate to the second order of the Taylor series expansion of $g(x)$ for any nonlinear function
- Advantages:
 - The EKF only calculates the posterior mean and covariance accurately to the first order with all higher order moments truncated; however, UT calculates the mean and covariance to the second order.
 - No need for continuous function as state or measurement function but any function can be used because projection through the non-linearity, no derivative taken.

More Information

- A cloud of 5000 samples drawn from a Gaussian prior is propagated through an arbitrary highly nonlinear function and the true posterior sample mean and covariance are calculated, which can be regarded as a ground truth of the two approaches, EKF and UT.



The scaling factor α

- As the dimension of the state-space increases, the radius of the sphere that bounds all the sigma points increases. Even though the mean and covariance of the prior distribution are still captured correctly, it does so at the cost of sampling non-local effects.
- These points are asymmetrically distributed about the mean. Higher order effects such as skew become more significant as the dimension increases.
- The sigma points can be scaled towards and away from the mean of the prior distribution by a proper choice of α : $\tau_i - \bar{x} \propto \sqrt{(n_x + \theta)}$

7.3. SCALED UNSCENTED TRANSFORMATION (SUT)

This improvement overcomes dimensional scaling effects by calculating the transformation of a scaled set of sigma points of the form $\tau_i = \tau_0 + \alpha (\chi_i - \chi_0)$, where α is a positive scaling parameter which can be made arbitrary small to minimize higher order effects.

The scaled unscented transformation can be written as:

$$S(W', x) \rightarrow S'(W', x')$$

$$x'_i = x_0 + \alpha (\chi_i - x_0)$$

$$y'_i = f[x'_i]$$

$$W'_i = \begin{cases} W_0 / \alpha^2 & i = 0 \\ W_i / \alpha^2 & i \neq 0 \end{cases}$$

$$\bar{y} = \sum_{i=0}^{2n} W'_i y'_i$$

$$P_y = \sum_{i=0}^{2n} W'_i (y'_i - \bar{y})(y'_i - \bar{y})^T + (1 + \beta - \alpha^2) (y_0 - \bar{y})(y_0 - \bar{y})^T$$

β is a parameter which minimizes the effects from high order terms

The sigma point selection and scaling can be combined into a single step by setting

$$\lambda = \alpha^2 (n_x + \theta) - n_x \text{ and selecting the sigma points are set by:}$$

$$x_0 = \bar{x}$$

$$\chi_i = \bar{x} + \sqrt{(n_x + \lambda) P_x} \quad i = 1, \dots, n_x$$

$$\chi_i = \bar{x} - \sqrt{(n_x + \lambda) P_x} \quad i = n_x + 1, \dots, 2n_x$$

$$W_0^{(a)} = \lambda / (n_x + \lambda)$$

$$W_0^{(c)} = \lambda / (n_x + \lambda) + (1 - \alpha^2 + \beta)$$

$$W_i^{(a)} = W_i^{(c)} = 1 / (2(n_x + \lambda)) \quad i = 1, \dots, 2n_x$$

7.4. UNSCENTED KALMAN FILTER

The unscented kalman filter (UKF) is a straightforward application of the scaled unscented transformation, where the state variable $x_t^a = [x_t^T v_t^T n_t^T]^T$ is an augmented vector and the covariance P_t^a is also an augmented matrix.

1. Initialization

$$x_0^a = \hat{E}[x_0^a] \quad P_0^a = \hat{E}[x_0^a x_0^a - x_0^a x_0^a]$$

$$\hat{x}_0^a = \hat{E}[x_0^a] = [\hat{x}_0^T \quad 0 \quad 0^T]$$

$$P_0^a = \hat{E}[(x_0^a - \hat{x}_0^a)(x_0^a - \hat{x}_0^a)^T] = \begin{bmatrix} P_0 & 0 & 0 \\ 0 & Q & 0 \\ 0 & 0 & R \end{bmatrix}$$

2. Updating

(a) Computing Sigma Points

$$\chi_{i,1} = \hat{x}_{i,1}^a \pm \sqrt{(n_x + \lambda) P_{i,1}^a}$$

b) Time update

$$\chi_{i,t+1} = f[\chi_{i,t}^a, x_{i,t}^a]$$

$$\bar{x}_{i,t+1} = \sum_{i=0}^{2n} W_i^{(a)} \chi_{i,t+1}$$

$$P_{i,t+1} = \sum_{i=0}^{2n} W_i^{(c)} [\chi_{i,t+1} - \bar{x}_{i,t+1}] [\chi_{i,t+1} - \bar{x}_{i,t+1}]^T$$

$$y_{i,t+1} = h[\chi_{i,t+1}^a, x_{i,t+1}^a]$$

$$\bar{y}_{i,t+1} = \sum_{i=0}^{2n} W_i^{(a)} y_{i,t+1}$$

c) Measurement Update

$$P_{i,t+1} = \sum_{i=0}^{2n} W_i^{(c)} [y_{i,t+1} - \bar{y}_{i,t+1}] [y_{i,t+1} - \bar{y}_{i,t+1}]^T$$

$$P_{i,t+1} = \sum_{i=0}^{2n} W_i^{(c)} [x_{i,t+1}^a - \bar{x}_{i,t+1}^a] [x_{i,t+1}^a - \bar{x}_{i,t+1}^a]^T$$

$$K_i = P_{i,t+1} P_{i,t+1}^{-1}$$

$$x_i = x_{i,t+1} + K_i (y_i - \bar{y}_{i,t+1})$$

$$P_i = P_{i,t+1} - K_i P_{i,t+1} K_i^T$$

$$\text{where, } x^a = [x^T \quad v^T \quad n^T]^T, \quad x^a = [x^T \quad v^T \quad n^T]^T$$

λ is composite scaling parameter, n_x, n_v, n_n, n_a

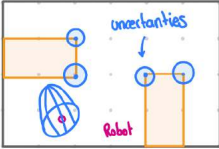
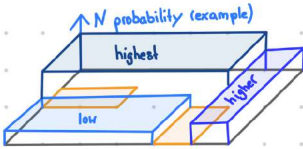
Q is process noise cov., R is measurement noise cov.

K is kalman gain, W is weights

8. Bayesian and Particle Filter Ch 9

Kalman filter is a bayesian filter with the constraint that the distribution is normal. Bayesian filter allows to have arbitrary distributions.

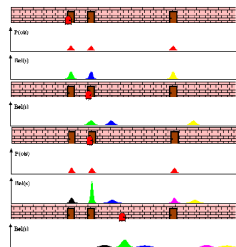
8.1. LOCALIZATION

Incremental localization	Global localization
- feature points (corners, ...) have uncertainties due to the uncertainty of the robot position and the sensors - uncertainty can be improved - uncertainty of position is n-dimensional gaussian - incremental localization: know at the beginning where we are, want to know how the robot moves → optimal tool to run SLAM: KF or EKF	- Global localization: initially no idea where the robot is - Initial distribution not gaussian, not smooth, - Could map probability distribution on a set of KF BUT better ways
	

- **Why not just use Kalman Filter?** Would be enough for incremental localization. Robot is not able to identify the door it has sensed → cannot associate an observation of a door uniquely to the doors given in the map → Data-Association problem → probabilistic localization

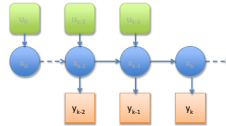
Probability properties

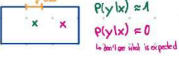
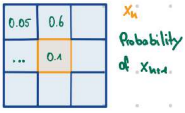
- $P(x, y) = P(x | y)P(y)$
- $P(x) = {}_yP(x, y) = {}_yP(x | y)P(y)$
- If x is independent of z given y : $P(x | y, z) = P(x | y)$
- Markov: $P(x_k | x_{k-1}, \dots, x_0) = P(x_k | x_{k-1})$



8.2. BAYESIAN FILTERING

→ probabilistically estimates a dynamic system's state (person's or object's location) from noisy updates



Given			Find
Sequence of measurements/ sensor readings (e.g. corners)	$y_1, \dots, y_k$	$\{y_{1:k}, x_{0:k-1}\}$	$P(x_k y_{1:k}, u_{0:k-1})$
Sequence of commands/control values (until last known step k-1)	$u_0, \dots, u_{k-1}$		Most likely state x at step k given a sequence of commands and measurements
Observation model/Sensor model/map	$P(y_k x_k)$	most likely sensor reading given state x Example: "Door in room" 	→ tells us how to use the sensory input to update the most recent estimate (prior) to obtain a new estimate (posterior)
Motion model/dynamic model	$P(x_{k+1} x_k)$	what are typical motions of the robot Example: 	
Prior probability	$P(x_0)$	See localization, uniform if unknown	

- Bayes Theorem: $P(x | y) = P(y | x)P(x)/P(y)$

- Think of x as the state of the robot and y as the data we know

$P(x_k | u_{0:k-1}, y_{1:k}) \rightarrow$ Posterior Probability Distribution

$$P(x_k | u_{0:k-1}, y_{1:k}) = P(y_k | x_k, u_{0:k-1}, y_{1:k-1}) P(x_k | u_{0:k-1}, y_{1:k-1}) / P(y_k | u_{0:k-1}, y_{1:k-1})$$

$$= \underbrace{\eta_k P(y_k | x_k)}_{\text{observation}} \underbrace{\int_{\mathcal{X}_{k-1}} P(x_k | u_{k-1}, x_{k-1}) P(x_{k-1} | u_{0:k-2}, y_{1:k-1}) dx_{k-1}}_{\text{state prediction recursive instance}}$$

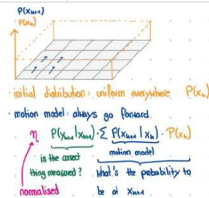
- Suppose instead state space is partitioned and probability in partition is constant

$$P(x | y) = P(y | x)P(x)/P(y) \approx P(x_i | y) = \eta P(y | x_i) P(x_i)$$

- Sum over all locations: $\eta = \sum_i P(y | x_i) P(x_i)$ (sum of resulting probability is 1)

- Thus, updating from observations is a simple multiplication of prior probability by likelihood of observation

$$P(x_{i,k} | u_{0:k-1}, y_{k-1}) = \sum_j P(x_{i,k} | u_{k-1}, x_{j,k-1}) P(x_{j,k-1} | u_{0:k-2}, y_{k-1})$$



- Thus, updating using dynamical model is simply a discrete convolution (blurring) of the prior by the driving noise of the planned motion

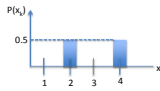
Motion

- Suppose we have $P(x_k)$ and $P(x_{k+1} | x_k, u_k) \rightarrow P(x_{k+1}) = P(x_{k+1} | x_k, u_k) P(x_k) dx_k$ (probability distribution for x_{k+1} given the command u_k and all the previous states x_k)

State space $X = \{1, 2, 3, 4\}$

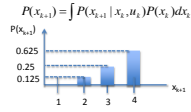
	$u_{k+1}=1$	$u_{k+1}=2$	$u_{k+1}=3$	$u_{k+1}=4$
$x_k=1$	0.25	0.5	0.25	0
$x_k=2$	0	0.25	0.5	0.25
$x_k=3$	0	0	0.25	0.75
$x_k=4$	0	0	0	1

Prior $P(x_k)$



Probability that robot is in state 4 in the next step

$$\begin{aligned}
 P(x_{k+1}=4) &= \sum_{x_k} P(x_{k+1}=4 | x_k, u_k) P(x_k) \\
 &= P(x_{k+1}=4 | x_k=1, u_k) P(x_k=1) + \\
 &= P(x_{k+1}=4 | x_k=2, u_k) P(x_k=2) + \\
 &= P(x_{k+1}=4 | x_k=3, u_k) P(x_k=3) + \\
 &= P(x_{k+1}=4 | x_k=4, u_k) P(x_k=4) \\
 &= 0 \times 0.5 + 0.25 \times 0.5 + 0.75 \times 0.25 + 1 \times 0.25 \\
 &= 0.625
 \end{aligned}$$



Discrete Bayes Filter Algorithm

Algorithm **Discrete_Bayes_filter**($u_{0:k-1}, y_{1:k}, P(x_0)$)

```

1.  $P(x) = P(x_0)$ 
2. for  $i=1:k$ 
3.   for all states  $x \in X$ 
4.      $P'(x) = \sum_{x'} P(x | u_{i-1}, x') P(x')$  Prediction given prior dist. and command
5.   end for
6.    $\eta = 0$ 
7.   for all states  $x \in X$ 
8.      $P(x) = P(y_i | x) P'(x)$  Update using measurement
9.      $\eta = \eta + P(x)$ 
10.  end for
11.  for all states  $x \in X$ 
12.     $P(x) = P(x) / \eta$  Normalize to 1
13.  end for
14. end for

```

	Kalman Filter	Multi-hypothesis tracking	Grid-based (fixed/variable)
Sensors	Gaussian	Gaussian	Non-Gaussian
Posterior	Gaussian	Multi-modal	Piecewise constant
Efficiency (memory)	++	++	+/-
Efficiency (time)	++	++	0/+
Implementation	+	0	+/0
Accuracy	++	++	+/++
Robustness	-	+	++
Global Localization	No	Yes	Yes

8.3. PARTICLE FILTERING

- Represent posterior/belief by a set of random samples instead of functions
- Estimation of non-Gaussian, nonlinear processes
- Examples: Monte Carlo filter, Survival of the fittest, Condensation, Bootstrap filter, Particle filter

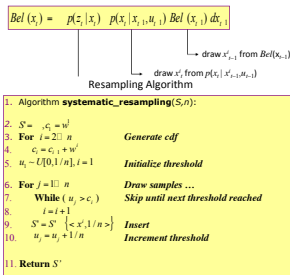
The Likelihood Function

- Generating the sensor likelihood is essentially a sensor simulation
 - can be expensive – pre-compute
 - approximate
- A good fast approximation is often a weighted sum of
 - a nominal model that is fast to compute
 - other deviations that are modeled as random elements

Resampling

- Given: Set S of weighted samples.
- Wanted: Random sample, where the probability of drawing x_i is given by w_i (sparse for low probability, dense for high probability)
- Typically done n times with replacement to generate new sample set S'

Particle Filter Algorithm

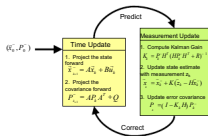
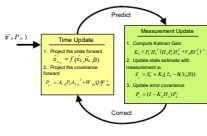


Also called **stochastic universal sampling**

Monte Carlo Localization

- Monte-Carlo-Localization(a, z, N, map)
 - $S = N$ samples from $P(X(t))$ from previous call
 - for** $i = 1$ **to** N
 - $S[i] = \text{sample from } P(X(t+1) | X(t) = S[i], A = a)$
 - $W[i] = 1$
 - for** $j = 1$ **to** M **do**
 - $z^* = \text{expected-sensor-reading}(S[i], \text{map})$
 - $W[i] = W[i] * P(z = z^* | z^* \neq z^*)$
 - $S = \text{weighted-sample-with-replacement}(N, S, W)$
 - return** S
- Note that S is a discrete representation of the probability of robot location

9. Overview Filters

	Kalman Filter	Extended Kalman Filter	Unscented Kalman Filter
Difference	KF requires a linear state transition and linear relation between state and measurement	EKF can deal with non-linear state transition and non-linear relation state-measurement through linear approximation by first-order Taylor expansion	UKF applies the scaled unscented transformation, which calculates the mean and covariance to the second order, thus can deal with highly non-linear problems
Prediction and update equation	 Time Update: 1. Propagate the state forward: $\hat{x}_k = A_k \hat{x}_{k-1} + B_k u_{k-1}$ 2. Propagate the covariance forward: $P_k = A_k P_{k-1} A_k^T + Q$ Measurement Update: 1. Compute Kalman Gain: $K_k = P_k H_k^T (H_k P_k H_k^T + R_k)^{-1}$ 2. Update state estimate with measurement z_k: $\hat{x}_k = \hat{x}_k + K_k (z_k - H_k \hat{x}_k)$ 3. Update error covariance: $P_k = (I - K_k H_k) P_k$	 Time Update: 1. Propagate the state forward: $\hat{x}_k = f(x_{k-1}, u_{k-1}, \Delta t)$ 2. Propagate the covariance forward: $P_k = A_k P_{k-1} A_k^T + W_k \Delta P W_k^T$ Measurement Update: 1. Compute Kalman Gain: $K_k = P_k H_k^T (H_k P_k H_k^T + R_k)^{-1}$ 2. Update state estimate with measurement z_k: $\hat{x}_k = \hat{x}_k + K_k (z_k - H_k \hat{x}_k)$ 3. Update error covariance: $P_k = (I - K_k H_k) P_k$	b) Time update $\hat{x}_{k+1} = f(\hat{x}_k, u_k)$ $\hat{x}_{k+1,1} = \sum_{i=0}^N W_i^{(m)} \hat{x}_{k+1,i}^{(m)}$ $P_{k+1} = \sum_{i=0}^N W_i^{(m)} (\hat{x}_{k+1,i}^{(m)} - \hat{x}_{k+1}) (\hat{x}_{k+1,i}^{(m)} - \hat{x}_{k+1})^T$ $\hat{x}_{k+1} = \hat{M}_{k+1} \hat{x}_k^*$ $P_{k+1} = \sum_{i=0}^N W_i^{(m)} P_{i,k+1}$ c) Measurement Update $P_{k+1} = \sum_{i=0}^N W_i^{(m)} (\hat{x}_{k+1,i}^{(m)} - \hat{x}_{k+1}) (\hat{x}_{k+1,i}^{(m)} - \hat{x}_{k+1})^T$ $P_{k+1} = \sum_{i=0}^N W_i^{(m)} (\hat{x}_{k+1,i}^{(m)} - \hat{x}_{k+1}) (\hat{x}_{k+1,i}^{(m)} - \hat{x}_{k+1})^T$ $K_k = P_k H_k^T (H_k P_k H_k^T + R_k)^{-1}$ $\hat{x}_k = \hat{x}_k + K_k (z_k - H_k \hat{x}_k)$ $P_k = P_k - K_k H_k P_k$ where: $x^{(m)} = [x^{(1)}, x^{(2)}, \dots, x^{(m)}]^T$ is measurement noise cov. R is measurement noise cov. K is Kalman gain, W is weights.
Name and explanation of methods	• K is the Kalman Gain	• H is the Jacobian matrix of partial derivatives of h with respect to x • V is the Jacobian matrix of partial derivatives of h with respect to measurement noise v • A is the Jacobian of f with respect to x • W is the Jacobian of f with respect to w	
Limits of applicability	KF, EKF and UKF require the error characteristic to modeled as zero-mean, Gaussian noise.	Cannot deal with highly non-linear models	
	Linear models (linear state transition and linear relation between state and measurement)		